DarkSolver Technical Overview

DarkSolver Research Notes

March 12, 2026

Abstract

DarkSolver is a simulation-first EVM auditing primitive that combines symbolic execution, protocol-aware objective generation, and replay-based validation. This note summarizes the state transition model, the storage abstraction strategy, and the Keccak-to-SMT mapping used in the repository's current public workflow.

1 Symbolic State

We model an execution state as

$$\Sigma = \langle pc, gas, stack, mem, stor, env, path, journal \rangle$$

where pc is the program counter, gas is the remaining gas budget, $stack$ and mem are EVM machine components, $stor$ is the symbolic storage view, env captures transaction and block context, $path$ is the accumulated path predicate, and $journal$ stores replay-relevant side effects.

For a bytecode image B and opcode $op = B[pc]$, DarkSolver applies a symbolic transition function

$$\delta : \Sigma \times op \to \mathcal{P}(\Sigma)$$

that returns zero, one, or multiple successor states depending on whether the opcode is terminal, deterministic, or branch-producing.

Fail-Closed Branching. When a branch cannot be modeled soundly with the available constraints, the engine treats it as a terminal failure instead of manufacturing a permissive successor. This design bias is used across stack underflow handling, malformed ABI decoding, and uncertain hydration responses.

2 Constraint Domain

The solver operates in Z3's quantifier-free bit-vector fragment:

`QF_ABV`

which matches EVM machine arithmetic more closely than floating-point or integer-relaxed encodings.

Representative arithmetic rules are expressed as fixed-width transformations:

$$\mathrm{ADD}_{256}(x, y) = (x + y) \bmod 2^{256}$$

$$\mathrm{MUL}_{512}(x, y) = \mathrm{zeroExtend}_{256}(x) \cdot \mathrm{zeroExtend}_{256}(y)$$

The second form is used in invariant comparisons where modular wrap-around would otherwise hide violations in reserve-product or basis-point drift checks.

3 Storage Abstraction

DarkSolver does not treat storage as a flat map alone. Instead it maintains:

1. a raw slot-addressed storage view,
2. a shadow structured view derived from SHA3 trace inference, and
3. projection constraints that connect structured keys to concrete hashed slots.

For a traced preimage $p = k \parallel s$ representing `keccak256(key . slot)`, the engine records the tuple

$$\tau = \langle p, h, n, pc \rangle$$

where h is the resulting hash term and n is the traced input width. Pattern inference then classifies traces into cases such as:

$$\texttt{FlatMapping}(slot) \qquad \texttt{NestedMapping}(slot, depth) \qquad \texttt{DynamicArray}(slot)$$

This allows reads and writes to attach symbolic meaning to storage accesses that would otherwise collapse into unrelated opaque keys.

4 Keccak Modeling

Direct bit-precise modeling of `keccak256` is generally too expensive for the intended audit workflow, so DarkSolver maps traced invocations into size-specialized uninterpreted functions:

$$K_{32}, K_{64}, K_{96}, K_{128} : \{0,1\}^{8n} \to \{0,1\}^{256}$$

For a 64-byte preimage p, the hash term is represented as

$$h = K_{64}(p)$$

while auxiliary projection constraints preserve relationships between concrete witnesses and symbolic keys. This gives the engine a usable compromise between scalability and structure recovery.

Interpretation Discipline. The uninterpreted-function layer is not treated as a proof of cryptographic behavior. It is instead a symbolic naming mechanism that preserves equality, disequality, and structured storage projections under bounded assumptions.

5 Objective Execution

The repository's objective catalog builds protocol-aware queries over hydrated state. Each objective is executed in an isolated solver context, producing one of:

$$\{\texttt{SAT}, \texttt{UNSAT}, \texttt{TIMEOUT}, \texttt{PANIC}\}$$

and the runner records per-objective timing and status for later audit review.

Candidate findings are admitted only after additional gates such as solvency checks, slippage sanity, or reserve-product consistency are applied. Formally, a candidate c is reportable only if

$$\mathrm{SAT}(c) \wedge G_{\text{solvency}}(c) \wedge G_{\text{price}}(c) \wedge G_k(c)$$

where the individual G terms denote invariant families implemented in the solver and verifier layers.

6 Replay Validation

DarkSolver's public workflow is simulation-only. After symbolic admission, a candidate path is replayed against pinned state with explicit block and timestamp context. The replay phase measures:

- execution success or revert classification,
- gas usage and failure step,
- mark-to-market balance deltas, and
- pricing coverage / stale oracle statistics.

This replay stage is what turns a satisfiable symbolic path into a reportable audit artifact.

7 Repository Positioning

The repository is intentionally structured as a research primitive:

- `src/` contains the core Rust implementation,
- `docs/` contains architecture and yellowpaper-style output,
- `benchmarks/` contains manifest-driven evaluation harnesses, and
- `reports/` contains example exploit writeups.

This separation keeps exploratory work visible without diluting the core engine surface.